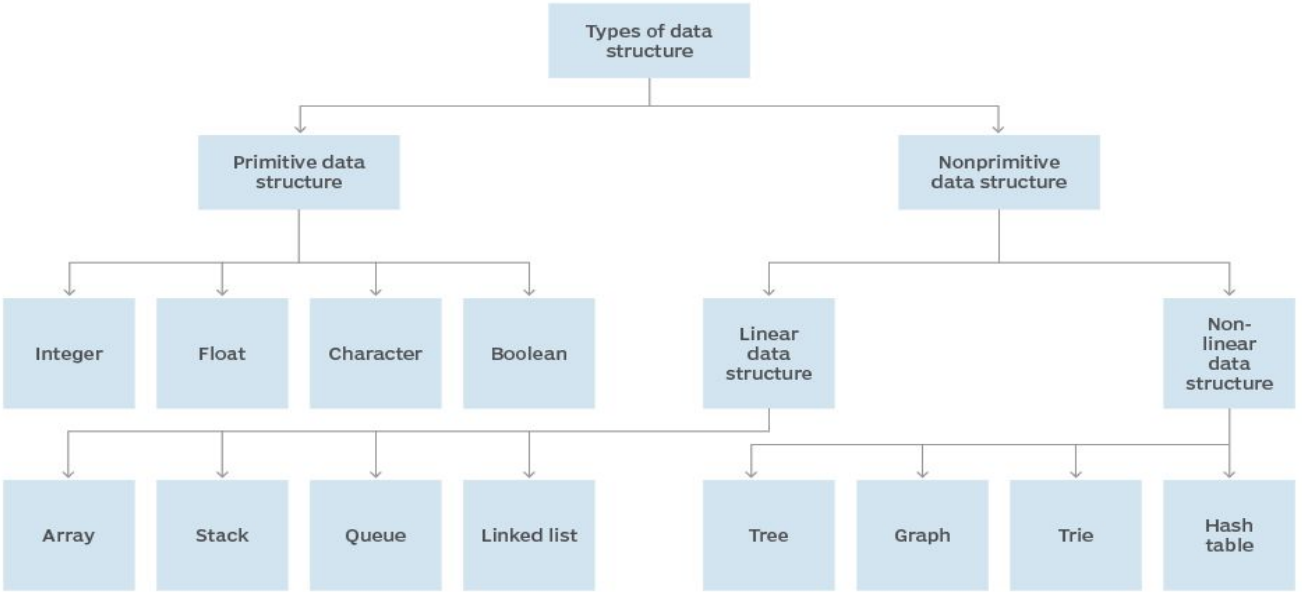
Data Structure

Padmaraj Nidagundi

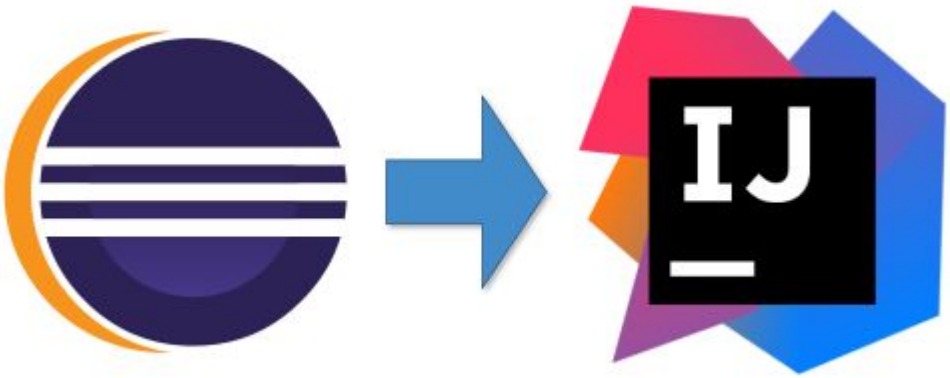
Last class.....

Tree

Data structure hierarchy



Student must need to use IDE

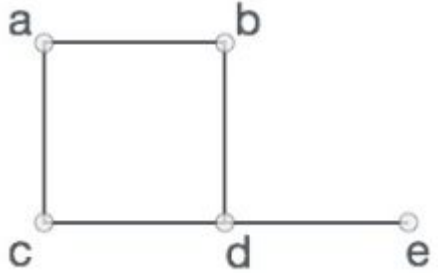


Light weight Online Java Compiler - https://www.codiva.io/
https://paiza.io/en/projects/new?language=java

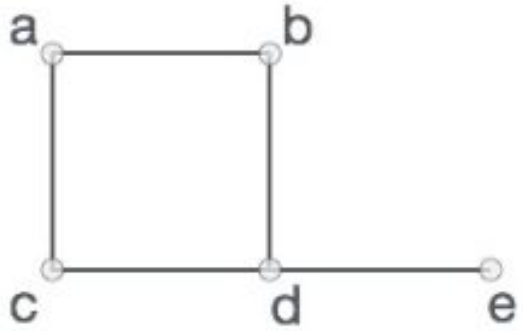
Today's Topic: Graphs

Graphs

A graph is a pictorial representation of a set of objects where some pairs of objects are connected by links.
The interconnected objects are represented by points termed as vertices, and the links that connect the vertices are called edges.



Formally, a graph is a pair of sets **(V, E)**, where **V** is the set of vertices and **E** is the set of edges, connecting the pairs of vertices. Take a look at the following graph −



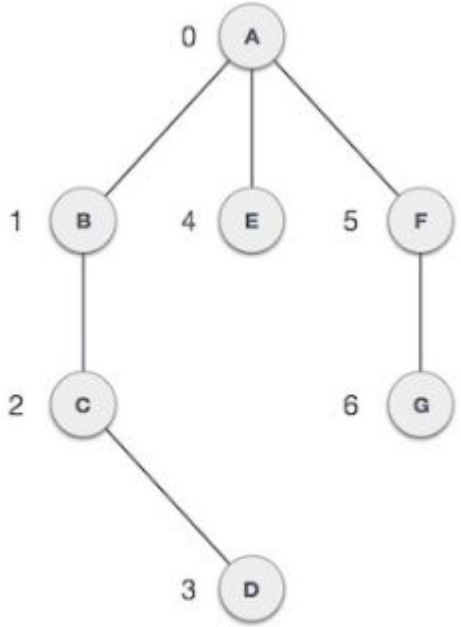
In the above graph,

V = {a, b, c, d, e}

E = {ab, ac, bd, cd, de}

Mathematical graphs can be represented in data structure. We can represent a graph using an array of vertices and a two-dimensional array of edges. Before we proceed further, let's familiarize ourselves with some important terms −

- **Vertex** − Each node of the graph is represented as a vertex. In the following example, the labeled circle represents vertices. Thus, A to G are vertices. We can represent them using an array as shown in the following image. Here A can be identified by index 0. B can be identified using index 1 and so on.
- **Edge** − Edge represents a path between two vertices or a line between two vertices. In the following example, the lines from A to B, B to C, and so on represents edges. We can use a two-dimensional array to represent an array as shown in the following image. Here AB can be represented as 1 at row 0, column 1, BC as 1 at row 1, column 2 and so on, keeping other combinations as 0.
- **Adjacency** − Two node or vertices are adjacent if they are connected to each other through an edge. In the following example, B is adjacent to A, C is adjacent to B, and so on.
- **Path** − Path represents a sequence of edges between the two vertices. In the following example, ABCD represents a path from A to D.



Basic Operations

Following are basic primary operations of a Graph −

- **Add Vertex** − Adds a vertex to the graph.
- **Add Edge** − Adds an edge between the two vertices of the graph.
- **Display Vertex** − Displays a vertex of the graph.

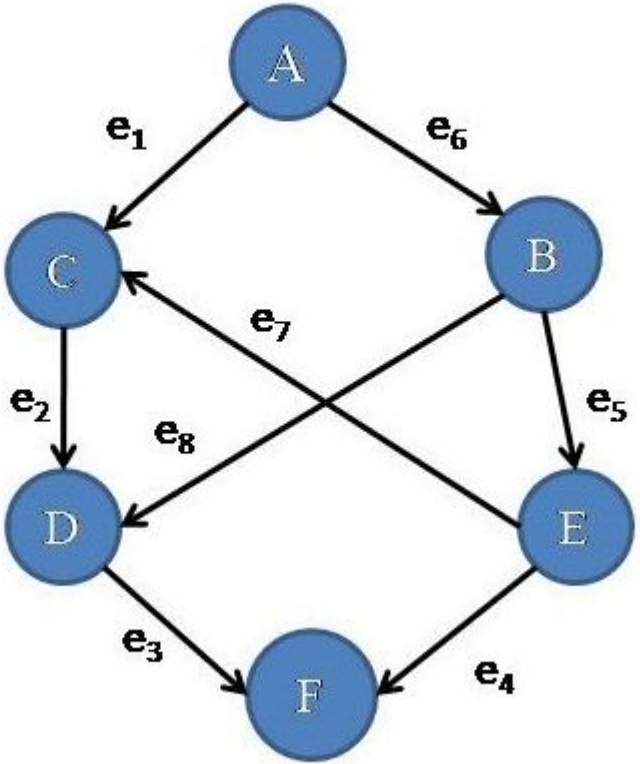
Graphs use in real life - advantages

- In maps that draw cities/states/regions as vertices and adjacency relations as edges.
- Once we have a graph of a particular concept, they can be easily used for finding shortest paths, project planning, etc.
- In circuit networks where points of connection are drawn as vertices and component wires become the edges of the graph.
- In transport networks where stations are drawn as vertices and routes become the edges of the graph.
- Graphs are also used to draw activity network diagrams. These diagrams are extensively used as a project management tool to represent the interdependent relationships between groups, steps, and tasks that have a significant impact on the project.
- Graphs are amazing data structures and we use them in everyday life. Facebook, Google, Uber,Quora they all use graph. By using graphs you can create any social network site. As persons represent to vertices while connection between them is edge. Graphs are easy to implement by using list or array. You have this flexibility to choose representation according to your requirement.

17 different types of a graph in data structur

- Directed Graph
- Undirected Graph
- Infinite Graph
- Trivial Graph
- Simple Graph
- Multi Graph
- Null Graph
- Complete Graph
- Pseudo Graph

1) Directed Graph: When all the edges have a direction from one node to another node then the graph is called Directed Graph. (Figure 1.2)



Home Task: Write 2 Graphs implementation example program

Examples here -

https://www.geeksforgeeks.org/implementing-generic-graph-in-java/
https://www.techiedelight.com/construct-directed-graph-from-undirected-graph/